

How you'll know it's working

Benchmarks, harnesses, contamination, and the cheap numbers you can afford every few thousand steps — evaluation as the instrument panel of a training run

Somewhere around the third week of your first real run, this will happen to you. Two checkpoints, five thousand steps apart. The loss went down — from 2.31 to 2.28, exactly as Chapter 6's budget said it should. Then someone runs "the benchmarks" and the numbers come back *worse*: MMLU dropped 1.4 points. Panic meeting. Rollback discussion. An afternoon of archaeology later, the truth: the second evaluation used a different prompt template — a colon where a period had been, one shot instead of five — and that formatting change alone was worth more points than five thousand steps of training. Nothing about the model had gotten worse; the *ruler* had changed length. This chapter is about building rulers that don't do that. It is the measurement discipline of Chapter 8 turned on quality instead of throughput: what a benchmark actually measures (a narrower thing than its name implies), how to run one so the number means the same thing next month, how to know your test set didn't leak into your training data, and — the part that pays rent daily — the cheap proxies you can afford to run *during* the run, so a two-week job never flies blind between full evaluations. Chapters 10 through 20 all end with "...and check the evals." This chapter is where those evals come from.

9.1 What benchmarks measure, and what they don't

Start with what actually happens when a model "takes" a benchmark, because the mechanics explain most of the surprises. Take a multiple-choice item — MMLU's format, four options, one correct¹. The model is not shown a form and asked to tick a box. The harness renders the question and each option into *text*, and asks the model for the log-probability of each candidate continuation:

```
# what "answering" an MMLU item actually is: four scored continuations
prompt = ""The strongest predictor of L2 pronunciation accuracy is:
A. age of acquisition
B. hours of formal study
C. first-language distance
D. motivation
Answer:""
# the harness scores log P(continuation | prompt) for each candidate:
#   " A"  -> -2.31          <- raw sum of token logprobs
```

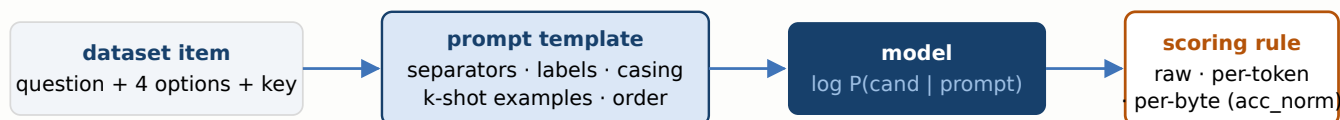
```
# " B" -> -2.87
# " C" -> -2.44
# " D" -> -3.05
# argmax says A. But candidates differ in length for full-text variants
# ("age of acquisition" vs "motivation"), so harnesses also report
# normalized scores (per-token, or per-BYTE: acc_norm) – and the two
# conventions can DISAGREE on the same item, and both are defensible.
```

Three consequences follow. First, a benchmark score is not "does the model know linguistics" — it is "under this prompt rendering, this shot count, and this normalization, how often is the correct option's continuation the likeliest." Change any of the three and you have a different instrument: the same model, on the same items, can swing by **up to 76 accuracy points** from formatting changes alone — separators, casing, option labels — in few-shot settings². That number, from careful controlled work on LLaMA-2-13B, should permanently end any instinct to compare scores across papers without checking configs. Second, generative benchmarks (GSM8K, code, IFEval) add a second instrument on top: the answer extractor — Chapter 25's regex lesson — plus decoding settings; "greedy vs. temperature 0.8" is worth points. Third, a benchmark is a *sample*: MMLU's 14K questions stand in for "knowledge," GSM8K's 1.3K for "arithmetic reasoning," and sampling error alone puts a ± 1 –2 point noise floor on many reported gaps (§9.7 makes this quantitative).

The two task species deserve names, because the whole chapter routes on the distinction. **Likelihood tasks** (MMLU, HellaSwag, ARC) never generate a token: they compare the probabilities of fixed continuations, which makes them cheap (one batched forward pass per candidate), deterministic given the config, and available from the first day of pre-training — a base model that has never seen an instruction can still rank continuations. **Generative tasks** (GSM8K, IFEval, anything with an extractor) sample an actual completion and grade it, which makes them slower, sensitive to decoding settings, dependent on the model following a format — and far closer to what your users experience. The practical rule that falls out: likelihood tasks are your *training-time* instruments, dense and stable enough to run on every checkpoint of a base model; generative tasks earn their keep from mid-training onward, and dominate once instruction-following exists (Chapter 23). A suite needs both, and §9.5's is built that way.

What benchmarks don't measure is the other half. They are static (the distribution your users generate drifts; the benchmark doesn't), they are narrow (four-option accuracy is a strange proxy for the open-ended generation your product ships), and they are targets (the moment a number drives decisions, Goodhart's law starts collecting rent — through contamination at worst, §9.3, and through unconscious overfitting of recipes to the suite at best). None of this makes benchmarks useless; it makes them *instruments with error bars and a calibration date*. The rest of this chapter is about using them that way.

WHAT A BENCHMARK SCORE IS MADE OF (one multiple-choice item, end to end)



THE SAME ITEM, TWO DEFENSIBLE ANSWERS

raw log-prob sum	byte-normalized (acc_norm)
"age of acquisition" -9.84 X long	"age of acquisition" -0.54/B ← wins
"motivation" -6.02 ← wins	"motivation" -0.60/B
short answers get a head start: fewer tokens, less log-prob to lose	length disadvantage removed — a different instrument, and a different accuracy number

every box above is a degree of freedom — a score without its template, shots, and normalization is a number without units.

Figure 9.1 — One item under the microscope. The score at the end depends on every stage, not just the model in the middle. The worked flip on the bottom row is illustrative, but the mechanism is real and routine — which is why harness configs, not just scores, must be reported.

9.2 Harness-style evaluation: lm-eval, and why you never hand-roll

Everything in Figure 9.1 that isn't the model is configuration, and configuration drifts unless it is pinned in code. That is the entire case for an evaluation harness, and it is why this book standardizes on EleutherAI's `lm-evaluation-harness` — the de facto community instrument, whose maintainers wrote up the reproducibility lessons the hard way.³ A harness pins the prompt template, the shot count and example selection, the normalization, the metric, and the dataset version, and emits them alongside the score. The discipline it enforces is the same one Chapter 8 demanded for MFU: *a number travels with its convention, or it doesn't travel*.

```
# the whole eval stack, pinned – CLI first:
# pip install lm_eval
lm_eval --model hf \
  --model_args pretrained=./ckpt-30000,dtype=bfloat16 \
  --tasks mmlu,hellaswag,arc_challenge,gsm8k \
  --num_fewshot 5 --batch_size auto \
  --log_samples --output_path results/step30000/
# results include the harness VERSION and full task configs – commit them.

# in-loop, from Python (the §9.4 cheap tier):
import lm_eval
res = lm_eval.simple_evaluate(
    model="hf", model_args={"pretrained": model},    # pass the live model
    tasks=["mmlu_stem_500", "hellaswag_es"],        # small custom slices
    num_fewshot=1, limit=500)                       # cap items for speed
log(step, {t: res["results"][t]["acc,none"] for t in res["results"]})
# full configs + custom task YAMLS: companion repo, eval/
```

Two habits make the harness actually deliver reproducibility rather than merely promise it. Pin the version — task templates get fixed and improved between releases, which silently moves scores; your suite should name an exact release the way it names a dataset revision, and a version bump should be a deliberate, diffed event (rerun one old checkpoint as a control). And log samples, not just aggregates — `--log_samples` stores every rendered prompt and every model response, which is what turns "GSM8K dropped two points" from a mystery into a diff: half the time the drop is thirty items that now phrase the answer in a way the extractor misses, which is a harness bug or a format drift, not a capability regression. The aggregate tells you *that*; the samples tell you *why* — and at the risk of repeating Chapter 25's §25.7: read them with eyes, weekly. Greps don't catch what eyes do.

A custom task is one YAML file

The harness's quiet superpower is that your own tasks — the Spanish slices, the private probe, a domain QA set — ride the same machinery as MMLU, with all the pinning for free. A task is a YAML file, and the whole Spanish held-out slice of §9.5 is this:

```
# eval/tasks/mmlu_es_500.yaml – a pinned custom task, end to end
task: mmlu_es_500
dataset_path: json                # or an HF dataset id
dataset_kwargs: {data_files: eval/data/mmlu_es_500.jsonl}
output_type: multiple_choice
doc_to_text: "{{question}}\nA. {{a}}\nB. {{b}}\nC. {{c}}\nD. {{d}}\nRespuesta:"
doc_to_choice: [" A", " B", " C", " D"]
doc_to_target: "{{answer}}"        # index of the correct option
num_fewshot: 1
metric_list:
  - {metric: acc, aggregation: mean}
  - {metric: acc_norm, aggregation: mean}
# committed next to the data file; the template IS the instrument.
# invoke: lm_eval --tasks mmlu_es_500 --include_path eval/tasks ...
```

Notice what living in this file buys. The template — down to the "Respuesta:" cue and the space before each option letter — is versioned text under review like any code, so §9.1's 76-point formatting hazard becomes a diffable artifact instead of ambient chaos. Both normalizations are computed every run, so a raw-vs-norm divergence (Figure 9.1's flip) is visible the day it starts. And when Chapter 10 asks "did the new mixture hurt Spanish," the answer comes from an instrument that has not moved since the day it was frozen.

9.3 Contamination: when the test set took the class

The nightmare scenario is quieter than you'd expect. Nobody cheats. A crawl from 2024 simply *contains* MMLU — the benchmark is on GitHub, on tutoring sites, in a thousand blog posts with worked answers — and Chapter 10's pipeline sweeps it in with everything else. The model then "solves" at evaluation time what it memorized at training time, your suite reads two points high, and every decision you calibrate against it (data mixtures, architecture choices, when to

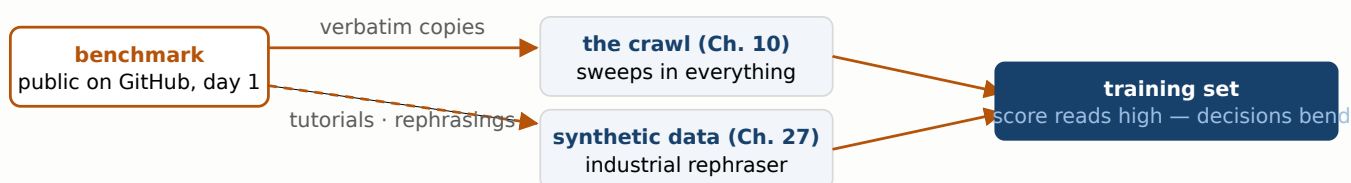
stop) inherits the bias. Reasoning benchmarks are the most-leaked category there is — Chapter 25 already flagged this for GSM8K and AIME — because worked solutions are exactly the content the internet loves to reproduce.

The standard defense is n-gram overlap, run in the direction people forget: you decontaminate the *training set* against the benchmarks, not the other way around. GPT-3's appendix set the pattern — flag any training document sharing a 13-gram with an eval item — and Llama 3's pipeline runs the same family of filters at scale⁴⁵. It is necessary, cheap relative to Chapter 10's pipeline, and *insufficient*: the LMSYS rephrasing study showed that a benchmark item paraphrased — same problem, new words — sails through n-gram and even embedding filters, and a 13B model fine-tuned on rephrased test sets matched GPT-4 on the corresponding benchmarks while having learned nothing general⁶. Paraphrase contamination is a special hazard of the synthetic-data era: Chapter 27's factories, if seeded carelessly, are industrial rephrasers of whatever was in their prompt context — never seed generation with benchmark items, and decontaminate synthetic output like any other data.

A fourth trick guards the other direction — your *own* held-out data leaking into your own training set through pipeline bugs rather than the internet: plant **canaries**. A canary is a unique nonsense string ("LATAM-EVAL-CANARY-7f3a9...") embedded in every private probe document; if the trained model can ever complete it at better than chance, your splits touched somewhere upstream, and no statistics are needed to prove it. BIG-bench institutionalized the pattern, and it costs nothing to adopt.⁷

So treat contamination control as three concentric rings. **Prevent**: n-gram + fuzzy decontamination of every training source against every benchmark you will ever report (the suite of §9.5, frozen early, makes "ever" tractable). **Detect**: keep a private held-out probe — a few hundred items you wrote yourselves, never published, in your product's domain and language — and watch the public-vs-private gap; a model that climbs on MMLU while flat on your private probe is telling you something. **Discount**: for public benchmarks on public crawls, assume residual leakage exists, and read small gaps (± 2 points) as weather, not signal. The labs' own reports now ship contamination analyses for exactly this reason — match that norm, and say what you filtered.

HOW THE TEST SET GETS INTO THE TRAINING SET (and what actually stops it)



THREE RINGS OF DEFENSE

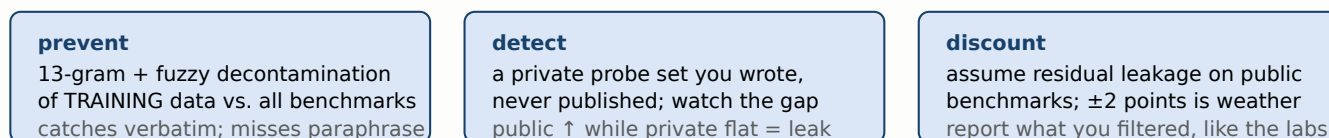


Figure 9.2 — Contamination is a pipeline property, not a sin. The leak paths run through ordinary crawling and — increasingly — through synthetic-data generation seeded with benchmark-adjacent text. No single defense suffices; the paraphrase pathway defeats n-grams by construction,⁶ which is what the private probe is for.

9.4 Cheap proxies you can afford every few thousand steps

The full suite of §9.5 takes GPU-hours you cannot spend every checkpoint, and this creates the instrument-panel problem: a two-week run with evaluations only at the ends is an aircraft with two windows, both painted over between takeoff and landing. The fix is a two-tier cadence. The expensive tier — the frozen suite — runs at real checkpoints, maybe daily. The cheap tier runs every few thousand steps, costs a minute or two, and exists to answer one question fast: *has anything gone wrong since the last look?*

Three proxies earn their place in the cheap tier, in order of cost. First, **held-out loss, sliced**. Not one perplexity — a fixed held-out shard per domain that matters: a FineWeb-Edu validation slice, a Spanish-text slice, a code slice. The aggregate loss can sit flat while the mixture quietly trades Spanish for code (Chapter 10's mixture experiments will do exactly this), and only per-domain curves show the trade. This is nearly free: a few hundred fixed batches, forward-only, and — Chapter 8's accounting — forward passes at eval batch sizes are cheap. Second, **small fixed slices of real benchmarks**: 500 items of MMLU at 1-shot, 500 of HellaSwag, run through the same harness with the same pinned config as the full suite so the numbers are comparable, just noisier. The role of these is trend, not level — a ± 2 -point wobble on 500 items is expected; a monotone three-reading slide is not. Third, **generation smoke tests**: a fixed file of twenty prompts — Spanish and English, instructions, a math item, a code stub — generated greedily and dumped to the log at every cheap-tier tick. This one is not a metric at all, and it catches what metrics miss: repetition loops, template leakage, a tokenizer bug mangling accents, the model suddenly answering in the wrong language. Thirty seconds of human reading per day, non-negotiable.

One warning about reading proxy curves: capability metrics and loss do not move in lockstep. Loss improves smoothly; a benchmark can sit at chance for a long stretch and then climb steeply — the "emergence" pattern, which careful analysis shows is often an artifact of hard, discontinuous metrics (exact match, accuracy) sitting on top of smoothly improving continuous ones (per-token likelihood)⁸. The practical translation: a flat GSM8K at step 40K is *not evidence the run is broken* while loss and per-domain perplexities improve on schedule — the sharp metric just hasn't crossed its threshold. This is also the argument for keeping likelihood-based proxies (loss on a fixed set of *correct reasoning traces*, say) alongside accuracy: the continuous shadow of the capability moves early and smoothly, and it will save you from killing healthy runs.

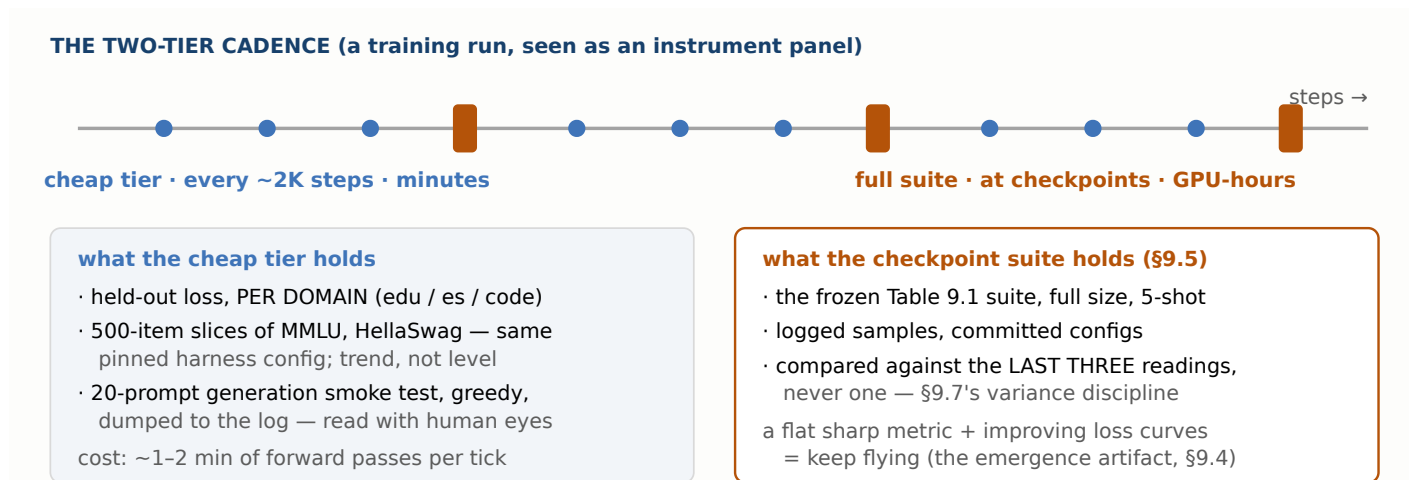


Figure 9.3 — Never fly blind between checkpoints. Blue dots are the cheap tier; amber bars the full suite. The cheap tier's job is anomaly detection — catching the mixture drift, the tokenizer bug, the divergence — within thousands of steps instead of days.

9.5 Building the suite the rest of this book uses

From here to Chapter 30, when a chapter says "and the evals held," it means the suite in Table 9.1. The design rules were fixed by the previous three sections: every task runs through the pinned harness with a committed config; the mix covers knowledge, commonsense, reasoning, code, and instruction-following, plus the languages this book's running example actually serves — Spanish is a first-class row, not an afterthought; every row carries its own noise bar, so a chapter claiming "+1.5 on GSM8K" can be checked against the ruler's own wobble; and the suite is *frozen* — additions are allowed, replacements are a versioned event with a rerun control, because a suite you keep editing is a ruler made of rubber.

Table 9.1 — The book's evaluation suite. Noise bars are the standard error of the mean at the stated size — the $\pm$ you should mentally attach before celebrating any gap. Runtimes: order-of-magnitude, one 7B model on one H100, likelihood tasks batched.

TASK	WHAT IT STANDS IN FOR	CONFIG	ITEMS	NOISE (± 1 SE)	RUNTIME
Held-out loss $\times$ 3 shards (edu / es / code)	the training objective itself, sliced	fixed batches, bf16	~2M tok	~0.003 nats	~2 min
MMLU ¹	breadth of knowledge	5-shot, acc	14,042	± 0.4 pts	~40 min
HellaSwag ⁹	commonsense continuation	0-shot, acc_norm	10,042	± 0.5 pts	~25 min
ARC-Challenge ¹⁰	grade-school science reasoning	25-shot, acc_norm	1,172	± 1.4 pts	~10 min

TASK	WHAT IT STANDS IN FOR	CONFIG	ITEMS	NOISE (± 1 SE)	RUNTIME
GSM8K ¹¹	multi-step arithmetic (generative)	5-shot, strict- match	1,319	± 1.3 pts	~35 min
IFEval ¹²	verifiable instruction- following	0-shot, prompt-level	541	± 2.1 pts	~15 min
Spanish block: Belebele- spa + MMLU-es slice	the languages we serve	matched configs	~2,400	± 1.0 pts	~15 min
Private probe (ours)	contamination detector + product domain	never published	~300	± 2.9 pts	~10 min

Read the noise column the way §9.7 will insist: with 1,319 items, GSM8K's standard error is ~1.3 points, so a 1-point "gain" between two checkpoints is a coin flip — and that is sampling noise alone, before prompt sensitivity and extraction quirks add their share. The suite's structural bets are worth making explicit. The three loss shards are the earliest and most sensitive instrument, and they are immune to most benchmark pathologies. The public benchmarks buy comparability with every model card ever published, at the price of the contamination discount. The Spanish block exists because a model whose training mixture is being tuned (Chapter 10) can bleed a language without the English aggregate noticing. And the private probe is doing double duty — contamination tripwire and the only row that measures your product's actual distribution. It is also, deliberately, the smallest and noisiest row: its job is detecting divergences, not resolving decimal points.

9.6 What the big labs do

Table 9.2 — Evaluation practice, as shipped.

LAB / EFFORT	PRACTICE	THE LESSON
Llama 3 ⁵	published per-benchmark prompt configs and decontamination method alongside scores; internal human evals for release gates	Configs are part of the result — copy the reporting norm
OLMo / OLMES ¹³	a fully specified open evaluation standard — templates, shots, normalization, curated in-loop suite during training	The whole instrument can be open-sourced; if in doubt, adopt theirs
lm-eval maintainers ³	documented how minor implementation details move scores across "the same" benchmark	Never compare numbers across different harnesses, period

LAB / EFFORT	PRACTICE	THE LESSON
HELM ¹⁴	many scenarios × many metrics (accuracy, calibration, robustness, bias) in one framework	Accuracy is one column, not the report card
DeepSeek-V3 ¹⁵	reports the eval framework per family of tasks, generative + likelihood mixed deliberately	Match the instrument to the task type, and say which you used

9.7 Practical considerations

Variance discipline, made mechanical. Three sources of wobble stack on every score: sampling error (the noise column of Table 9.1 — shrink it only with more items), prompt-format sensitivity (up to tens of points in adversarial cases² — pin templates and never change them mid-run), and run-to-run nondeterminism (batching, kernels, and precision genuinely change logprobs near ties; bf16 vs fp16 evaluation of the same checkpoint differs). The working rules: report the config with the number; compare a checkpoint against the *trend of the last three*, never one reading; for generative tasks with sampling, avg@k over fixed seeds (Chapter 25's rule generalizes); and before trusting any gap, ask whether it clears *two* noise bars, because you are usually differencing two noisy numbers.

What we measured, so you don't have to. On our own suite, re-evaluating the identical checkpoint moved likelihood benchmarks by a few tenths of a point when we changed evaluation batch size (different padding and kernel paths near probability ties), by up to half a point switching bf16 to fp32 logprob computation, and by two full points on GSM8K when a harness upgrade changed the answer extractor's tolerance for "1,000" versus "1000". None of these are model changes; all of them are the reason §9.5 freezes the stack down to the version number and the reason a gap smaller than two noise bars earns a shrug. The one we now check first after any infra change: rerun *one fixed old checkpoint* and confirm the suite reproduces itself before believing anything about the new one.

The chat-template cliff. The single most common "our model got worse" false alarm after SFT (Chapter 23) is evaluating a chat model with a base-model harness config or vice versa — the score cliff is real, but it measures template mismatch, not capability. Decide per checkpoint stage which rendering the harness uses, and log it. Related: likelihood tasks on models with added special tokens need the same tokenizer the training loop used, byte for byte — Chapter 11's fertility surprises apply to eval prompts too.

Common pitfalls

Comparing across harnesses or versions.

"We match Llama 3 on MMLU" means nothing unless the template, shots, and normalization match. Reproduce the baseline under YOUR pinned harness before comparing.

Editing the suite mid-run.

A new task version, a "better" template — and every historical curve breaks. Freeze; version bumps rerun an old checkpoint as control.

Celebrating inside the noise bar.

+0.8 on a 1.3-SE benchmark is a coin flip. Two noise bars or it didn't happen.

Skipping the smoke test.

Metrics stayed flat while generations degenerated into repetition — it happens, especially near instability (Ch. 18). Twenty prompts, human eyes, daily.

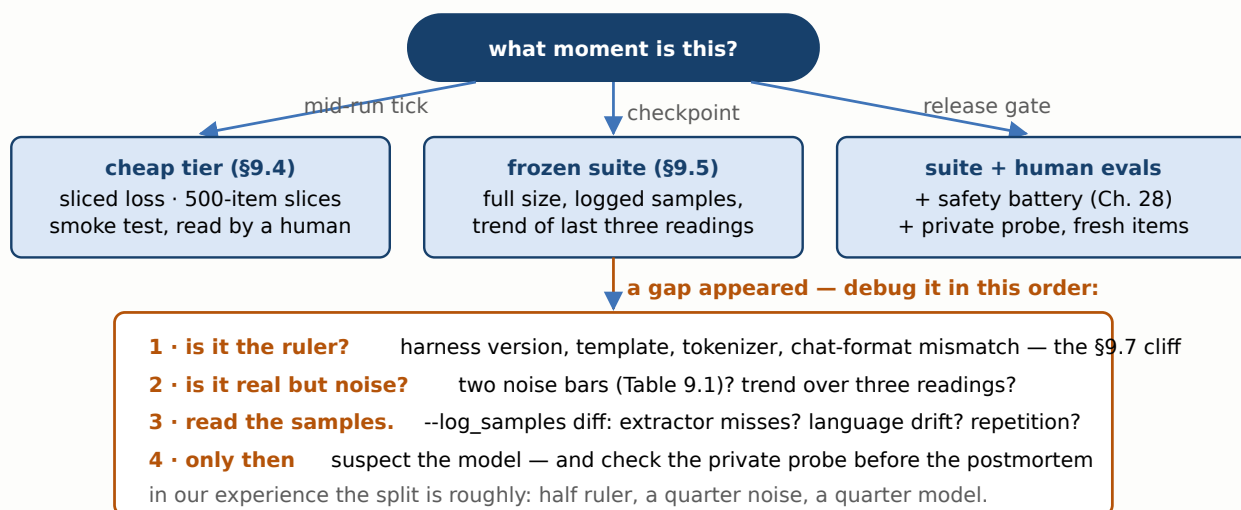
Decontaminating the benchmark instead of the data.

Filtering eval items "seen in training" quietly makes your test easier over time. The training set is what gets filtered.

Trusting one number after a big recipe change.

Packing (Ch. 16), a new mixture (Ch. 10), a context extension (Ch. 21) — big changes deserve the full suite plus sample reading, not the cheap tier's 500-item slice.

9.8 Decision framework: what to run, and when



cost-benefit: the cheap tier is ~1% of compute and catches most disasters; the frozen suite is the contract.

Figure 9.4 — What to run, when — and the debug order when a number moves. The order in the amber box is by base rates: most "regressions" die at step 1 or 2, which is exactly why the steps are ordered this way.

Cost-benefit, bluntly. The whole apparatus of this chapter costs a few percent of a run: the cheap tier ~1% of compute, the checkpoint suite a few GPU-hours a day, the private probe a few team-days once. Against that: a single decision made on a contaminated or misconfigured number — a mixture chosen wrong, a healthy run killed, a regression shipped — costs more than a year of evaluation compute. **How this scales:** on one node, the cheap tier and a nightly suite are plenty; at Chapter 14's scale, evals get their own small pool so they never queue behind training; at Chapter 29's scale, evaluation becomes a team — because every chapter downstream of this one, from data mixtures to RL, is steering by these numbers, and the steering is only as good as the instruments.

9.9 Summary

A benchmark score is a measurement made by an instrument with many settings — template, shots, normalization, extractor, dataset version — and it means nothing detached from them; the same model can move tens of points on formatting alone. So pin everything in a harness, commit the configs, log the samples, and let every number travel with its convention. Guard the instrument against contamination in three rings — decontaminate the training data (n-grams catch verbatim, not paraphrase), keep a private probe you never publish, and discount small public-benchmark gaps — and guard your judgment against noise by reading trends over three checkpoints and demanding two standard errors before believing a gap. Between the expensive checkpoint evaluations, fly on the cheap tier: per-domain held-out loss, small pinned benchmark slices, and a twenty-prompt smoke test read by human eyes — a minute of compute that turns a blind two-week run into an instrumented one. The suite of Table 9.1 — knowledge, commonsense, reasoning, code-adjacent instruction following, Spanish as a first-class row, and one private probe — is frozen from here on: every recipe in Parts III through V is judged against it, which is the point. Training is a sequence of decisions under uncertainty; this chapter is the instrument panel those decisions steer by, and Chapter 10 starts using it immediately.

References

1. Hendrycks, D., et al. (2021). Measuring Massive Multitask Language Understanding (MMLU). *ICLR*. arXiv:2009.03300. 14,042 four-option questions across 57 subjects; the de facto knowledge benchmark.
2. Sclar, M., et al. (2024). Quantifying Language Models' Sensitivity to Spurious Features in Prompt Design. *ICLR*. arXiv:2310.11324. Up to 76 accuracy points of spread from semantically equivalent formatting changes (LLaMA-2-13B, few-shot).
3. Biderman, S., et al. (2024). Lessons from the Trenches on Reproducible Evaluation of Language Models. arXiv:2405.14782. The lm-evaluation-harness maintainers on why implementation details move scores, and the reporting discipline that follows.
4. Brown, T., et al. (2020). Language Models are Few-Shot Learners (GPT-3). *NeurIPS*. arXiv:2005.14165. Appendix C: 13-gram overlap decontamination of training data against benchmarks — the pattern the field adopted.
5. Grattafiori, A., et al. (2024). The Llama 3 Herd of Models. arXiv:2407.21783. Scaled n-gram-family decontamination; per-benchmark evaluation configs published with scores.
6. Yang, S., et al. (2023). Rethinking Benchmark and Contamination for Language Models with Rephrased Samples. arXiv:2311.04850. Paraphrased test sets evade n-gram and embedding filters; a 13B fine-tuned on them matches GPT-4 on the corresponding benchmarks.
7. Srivastava, A., et al. (2022). Beyond the Imitation Game (BIG-bench). arXiv:2206.04615. Introduced canary strings for detecting benchmark data in training corpora.
8. Schaeffer, R., Miranda, B., & Koyejo, S. (2023). Are Emergent Abilities of Large Language Models a Mirage? *NeurIPS*. arXiv:2304.15004. Discontinuous metrics over smoothly improving likelihoods produce apparent emergence; metric choice, not capability, drives much of the "jump."
9. Zellers, R., et al. (2019). HellaSwag: Can a Machine Really Finish Your Sentence? *ACL*. arXiv:1905.07830.

10. Clark, P., et al. (2018). Think you have Solved Question Answering? Try ARC. arXiv:1803.05457.
11. Cobbe, K., et al. (2021). Training Verifiers to Solve Math Word Problems (GSM8K). arXiv:2110.14168.
12. Zhou, J., et al. (2023). Instruction-Following Evaluation for Large Language Models (IFEval). arXiv:2311.07911. 25 types of programmatically verifiable instructions; 541 prompts.
13. Gu, Y., et al. (2024). OLMES: A Standard for Language Model Evaluations. arXiv:2406.08446. Fully specified open evaluation standard — templates, shots, normalization — adopted by the OLMo effort.
14. Liang, P., et al. (2022). Holistic Evaluation of Language Models (HELM). arXiv:2211.09110. Scenarios $\times$ metrics beyond accuracy: calibration, robustness, fairness, efficiency.
15. DeepSeek-AI (2024). DeepSeek-V3 Technical Report. arXiv:2412.19437. Per-task-family evaluation frameworks reported alongside results.